

SKILL-3 WEEK

Task-1

TO Apply Single Quotes, Preserve Literal Content, Ignore Variable Expansion, Store Quoted Strings, Validate Parsing Results, Test Edge Cases

```
GNU nano 8.7.1 task5.c *
#include <stdio.h>
#include <string.h>

#define MAX_INPUT 500
#define MAX_OUTPUT 500

void parseSingleQuotes(char *input, char *output) {
    int i = 0;
    int j = 0;
    int inSingleQuote = 0;

    while (input[i] != '\0') {
        if (input[i] == '\\') {
            inSingleQuote = !inSingleQuote;
            i++;
            continue;
        }

        output[j++] = input[i];
        i++;
    }

    output[j] = '\0';
}

int validateQuotes(char *input) {
    int singleQuoteCount = 0;

    for (int i = 0; input[i] != '\0'; i++) {
        if (input[i] == '\\') {
            singleQuoteCount++;
        }
    }

    if (singleQuoteCount % 2 != 0) {
        return 0;
    }

    return 1;
}

int main() {
    char input[MAX_INPUT];
    char output[MAX_OUTPUT];

    printf("=====\n");
}
```


Output:

```
root@DESKTOP-QU0HVJI:~# nano task5.c
root@DESKTOP-QU0HVJI:~# gcc -Wall task5.c -o task5
root@DESKTOP-QU0HVJI:~# ./task5
=====
SINGLE QUOTE PARSER
=====

Enter a command:
shell> echo 'hello world'

Original Input : echo 'hello world'
Parsed Output  : echo hello world

Single quotes are treated as literal content.
Variable expansion is not performed inside single quotes.
root@DESKTOP-QU0HVJI:~# |
```

Report: In this task, I implemented handling of single quotes in a shell command using C. The text inside single quotes was treated as literal content, so spaces and special characters were preserved. Variable names such as \$HOME were not expanded. The program also checked whether the single quotes were properly closed and displayed an error for unmatched quotes. Different inputs were tested to verify the parsing results.

Task-2

To Apply Double Quotes, Preserve Spaces, Allow Variable Expansion, Parse Nested Tokens, Validate Outputs, Test Quoted Commands

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

#define MAX_INPUT 500
#define MAX_OUTPUT 1000

/* Check whether double quotes are balanced */
int validateQuotes(char *input) {
    int count = 0;

    for (int i = 0; input[i] != '\0'; i++) {
        if (input[i] == '"')
            count++;
    }

    return count % 2 == 0;
}

/* Expand environment variables */
void expandVariable(char *input, int *i, char *output, int *j) {
    char variable[100];
    int k = 0;

    (*i)++; // Skip $

    while (input[*i] != '\0' &&
           (isalnum(input[*i]) || input[*i] == '_')) {

        if (k < 99)
            variable[k++] = input[*i];

        (*i)++;
    }

    variable[k] = '\0';

    char *value = getenv(variable);

    if (value != NULL) {
        strcpy(&output[*j], value);
        *j += strlen(value);
    } else {
        output[(*j)++] = '$';
    }
}
```

Output:

```
root@DESKTOP-QU0HVJI: ~/osp$ nano task6.c
root@DESKTOP-QU0HVJI: ~/osp$ gcc -Wall task6.c -o task6
root@DESKTOP-QU0HVJI: ~/osp$ ./task6
=====
DOUBLE QUOTE PARSER
=====

Enter a command:
shell> echo "User is $USER"

Original Input : echo "User is $USER"
Parsed Output  : echo User is pavani

Double quotes preserve spaces.
Variables beginning with $ are expanded.
root@DESKTOP-QU0HVJI:~/osp$
```

Report: In this task, I implemented double quote handling in C. Double quotes were used to preserve spaces within a command while allowing environment variables such as \$USER and \$HOME to be expanded. The program checked for unmatched double quotes and displayed the parsed output. Different quoted commands were tested to verify that spaces and variables were handled correctly.

Session-6

Task-1

To Create Process Escape Sequences, Handle Escaped Spaces, Escape Special Symbols, Preserve Characters, Validate Parser Output, Test Complex Inputs

```
GNU nano 8.7.1 task7.c *
#include <stdio.h>
#include <string.h>

#define MAX_INPUT 500
#define MAX_OUTPUT 500

void parseEscapes(char *input, char *output) {
    int i = 0;
    int j = 0;

    while (input[i] != '\0') {
        if (input[i] == '\\') {
            i++;

            if (input[i] != '\0') {
                output[j++] = input[i];
                i++;
            }
        } else {
            output[j++] = input[i];
            i++;
        }
    }

    output[j] = '\0';
}

int main() {
    char input[MAX_INPUT];
    char output[MAX_OUTPUT];

    printf("=====\n");
    printf("    ESCAPE SEQUENCE PARSER\n");
    printf("=====\n");

    printf("\nEnter a command:\n");
    printf("shell> ");

    fgets(input, MAX_INPUT, stdin);

    input[strcspn(input, "\n")] = '\0';

    parseEscapes(input, output);

    printf("\nOriginal Input : %s\n", input);
    printf("Parsed Output  : %s\n", output);
}
```

OUTPUT:

```
root@DESKTOP-QU0HVJI:~# nano task7.c
root@DESKTOP-QU0HVJI:~# gcc -Wall task7.c -o task7
root@DESKTOP-QU0HVJI:~# ./task7
=====
          ESCAPE SEQUENCE PARSER
=====

Enter a command:
shell> echo hello\ world

Original Input : echo hello\ world
Parsed Output  : echo hello world

Escape sequences were processed successfully.
```

Report: In this task, I implemented escape sequence handling using the backslash character. Escaped spaces and special characters were processed so that they were treated as normal characters. The backslash was removed during parsing while the escaped character was preserved. Different inputs containing spaces, special symbols, and variables were tested to check the parser output.

Task-2: To Create Child Processes, Execute Programs, Handle Execution Errors, Pass Arguments, Manage Parent Process, Test Command Launching

```
GNU nano 8.7.1 task8.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid;
    int status;

    printf("=====\n");
    printf("    PROCESS CREATION AND EXECUTION\n");
    printf("=====\n");

    printf("\nParent process started.");
    printf("\nParent PID: %d\n", getpid());

    /* Create child process */
    pid = fork();

    if (pid < 0) {
        perror("fork failed");
        return 1;
    }

    /* Child process */
    if (pid == 0) {

        printf("\nChild process created.");
        printf("\nChild PID: %d\n", getpid());

        printf("Executing: ls -l\n\n");

        /* Execute program with arguments */
        execlp("ls", "ls", "-l", NULL);

        /* This executes only if exec fails */
        perror("exec failed");
        exit(1);
    }

    /* Parent process */
    else {

        printf("\nParent is waiting for child...\n");

        if (waitpid(pid, &status, 0) == -1) {
            perror("waitpid failed");
            return 1;
        }
    }
}
```

OUTPUT:

```
root@DESKTOP-QU0HVJI: ~#  
Escape sequences were processed successfully.  
root@DESKTOP-QU0HVJI:~# nano task8.c  
root@DESKTOP-QU0HVJI:~# gcc -Wall task8.c -o task8  
root@DESKTOP-QU0HVJI:~# ./task8  
=====
```

PROCESS CREATION AND EXECUTION									
Parent process started.									
Parent PID: 925									
Parent is waiting for child...									
Child process created.									
Child PID: 926									
Executing: ls -l									
total 248									
-rw-r--r--	1	root	root	0	Aug	4	05:07	LICENSE	
-rw-r--r--	1	root	root	0	Aug	4	05:07	Makefile	
-rw-r--r--	1	root	root	0	Aug	4	05:07	README.md	
drwxr-xr-x	2	root	root	4096	Aug	4	05:09	assets	
drwxr-xr-x	2	root	root	4096	Aug	4	05:07	bin	
drwxr-xr-x	2	root	root	4096	Aug	4	05:08	docs	
-rwxr-xr-x	1	root	root	16088	Aug	4	05:31	fork_example	
-rw-r--r--	1	root	root	443	Aug	4	05:36	fork_example.c	
-rwxr-xr-x	1	root	root	16040	Aug	4	05:34	fork_exec	
-rw-r--r--	1	root	root	389	Aug	4	05:34	fork_exec.c	
drwxr-xr-x	2	root	root	4096	Aug	4	05:08	include	
drwxr-xr-x	2	root	root	4096	Aug	4	05:07	obj	
drwxr-xr-x	2	root	root	4096	Aug	4	05:08	scripts	
drwxr-xr-x	2	root	root	4096	Aug	4	05:07	src	
-rwxr--r--	1	root	root	19072	Aug	12	08:46	task2	
-rw-r--r--	1	root	root	3976	Aug	12	08:46	task2.c	
-rwxr--r--	1	root	root	16448	Aug	12	08:50	task3	
-rw-r--r--	1	root	root	5425	Aug	12	08:50	task3.c	
-rwxr--r--	1	root	root	26872	Aug	12	08:53	task4	
-rw-r--r--	1	root	root	9401	Aug	12	08:53	task4.c	
-rwxr--r--	1	root	root	16256	Aug	12	09:09	task5	
-rw-r--r--	1	root	root	1527	Aug	12	09:09	task5.c	
-rwxr--r--	1	root	root	16472	Aug	12	09:12	task6	
-rw-r--r--	1	root	root	2349	Aug	12	09:12	task6.c	
-rwxr--r--	1	root	root	16208	Aug	12	09:14	task7	
-rw-r--r--	1	root	root	1041	Aug	12	09:14	task7.c	
-rw-r--r--	1	root	root	16304	Aug	12	09:17	task8	
-rw-r--r--	1	root	root	1359	Aug	12	09:17	task8.c	
-rw-r--r--	1	root	root	4096	Aug	4	05:08	tests	

```
Child exited with status: 0  
Parent process completed.  
root@DESKTOP-QU0HVJI:~#
```

Report: In this task, I implemented process creation and program execution using `fork()`, `exec()` and `waitpid()`. The parent process created a child process, and the child executed the `ls` command with arguments using `execlp()`. The parent waited for the child process to finish and checked its exit status. Error handling was also implemented for cases where the command could not be executed. The program successfully demonstrated process creation, execution, argument passing, and parent-child process management.

